

Waypoint: Codebase & Architecture Guide

An in-depth, beginner-friendly walkthrough explaining every file, directory, technical dependency, and architectural concept powering the Waypoint academic & career navigation platform.

Stack: Next.js 14 + React Flow + FastAPI + Scikit-Learn

Platform: Waypoint Academic & Career Navigator

Audience: Beginner to Full-Stack Developer

1. The Big Picture: How Waypoint Works

THE RESTAURANT ANALOGY (FOR ABSOLUTE BEGINNERS)

If Waypoint were a world-class restaurant:

- **frontend/ is the Dining Room:** The beautiful menus, tables, lighting, and buttons that the customer sees, clicks, and touches.
- **backend/ is the Kitchen:** The chefs, ovens, and algorithms doing the heavy math, graph calculations, and AI logic behind closed doors.
- **data/ is the Pantry:** The raw ingredients (spreadsheets with skills, prerequisites, and job trends).
- **tests/ is the Quality Inspector:** Automated tests making sure recipes work without failing.
- **docs/ is the Architectural Blueprint:** Explaining the rules and design of the platform.

System Architecture Flow



2. Complete Technology Stack & Libraries

Here are all the frameworks, libraries, and tools used to build Waypoint, and why each was chosen:

Frontend Technologies

Technology	Category	What It Does in Waypoint
Next.js 14 (App Router)	Full-Stack Framework	Provides file-system routing (e.g. <code>app/dashboard</code> , <code>app/route/[id]</code>), server-side rendering, and performance optimizations.
React 18 & React-DOM	UI Library	Powers modular components, state management (<code>useState</code> , <code>useEffect</code>), and custom hooks.
React Flow (reactflow)	Graph Visualization	The core interactive roadmap canvas. Allows students to pan, zoom, inspect skill nodes, and trace prerequisite paths visually.
TypeScript	Type Safety	Enforces strict data contracts for routes, skills, and student profiles, catching bugs before runtime.
Tailwind CSS	Styling	Utility-first CSS framework for clean, modern cards, buttons, badges, and responsive layouts without custom CSS bloat.
Framer Motion	Animation	Handles smooth page transitions, sliding skill detail drawers, and interactive card hovering effects.
Lucide React	Iconography	Clean, accessible iconography (compass, lock, checkmarks, trending arrows) used across navigation and roadmap nodes.
Recharts	Data Visualization	Renders charts for skill gap distributions, market demand signals, and progress tracking.

Backend & Data Technologies

Technology	Category	What It Does in Waypoint
FastAPI (Python 3.11)	Backend Web Framework	Asynchronous Python REST API framework delivering lightning-fast endpoints for route calculations and skill audits.
Uvicorn	ASGI Server	High-performance server hosting the FastAPI application.
Pydantic v2	Validation & Schemas	Validates incoming API payloads and serializes responses with strict type checking.
Pandas & NumPy	Data Science	Loads, filters, and joins the CSV datasets (skills, prerequisites, historical student trails).
Scikit-learn	Machine Learning	Provides K-Means clustering and vector similarity to cluster synthetic student journeys into "Crowd Trails".

Technology	Category	What It Does in Waypoint
Google Generative AI (Gemini)	Generative AI	Turns deterministic mathematical graph outputs into natural, encouraging career advice and learning step rationales.

3. File-by-File & Folder-by-Folder Guide

A. Root Directory Files

File / Directory	Plain-English Explanation
<code>.devcontainer/devcontainer.json</code>	Cloud Dev Environment: Tells GitHub Codespaces or Docker to pre-install Python 3.11, Node 20, and all VS Code tools automatically.
<code>requirements.txt</code>	Python Package List: Specifies all required backend dependencies (FastAPI, Scikit-learn, Pandas, Gemini SDK).
<code>.env.example</code>	Secrets Template: Demonstrates required environment variables (e.g. <code>GEMINI_API_KEY</code> , <code>DATABASE_URL</code>) without exposing credentials.
<code>.gitignore</code>	Exclusion List: Instructs Git to ignore build folders (<code>node_modules</code> , <code>.next</code> , <code>__pycache__</code>).
<code>README.md</code>	Project Documentation: Welcome page, system overview, architecture diagram, and setup quickstart.

B. `data/` Directory (The Curated Datasets)

File	What It Contains & How It Is Used
<code>skills.csv</code>	List of skills (ID, name, category, difficulty) mapped to career tracks like AI/ML Engineer and Backend Developer.
<code>prerequisites.csv</code>	The Dependency Graph: Directed pairs (e.g. <code>FastAPI -> Python</code>) with dependency strength ratings (0.0 to 1.0).
<code>student_trails.csv</code>	Historical student learning trajectories used by Machine Learning algorithms to find popular, successful learning paths.
<code>market_signals.csv</code>	Labor market indices (e.g., Python demand score 95, rising trend) used to recommend high-value skills.
<code>opportunities.csv</code>	Internships and scholarships tagged with prerequisite requirements, enabling personalized opportunity matching.

C. `frontend/app/` Directory (The Web Pages)

Page Path	What the User Sees & Experiences
<code>app/layout.tsx</code>	Global Layout: Embeds the root HTML, Google Fonts, and wraps the app in the global <code>AuthProvider</code> .
<code>app/page.tsx</code>	Landing Page (/): Welcome hero section, interactive route preview, feature showcases, and sign-up prompts.

Page Path	What the User Sees & Experiences
app/onboarding/page.tsx	Onboarding Wizard (/onboarding) : Interactive 4-step questionnaire (Profile, Academic Background, Current Skills, Career Destination).
app/dashboard/page.tsx	Student Dashboard (/dashboard) : Central hub with circular progress rings, active milestones, next recommended skills, and route summaries.
app/routes/page.tsx	Routes Directory (/routes) : Displays alternative learning routes (Fast Track, Balanced, Foundation First) with duration and workload metrics.
app/route/[id]/page.tsx	Interactive Route Map (/route/balanced) : Fullscreen React Flow DAG roadmap where students pan, zoom, and inspect prerequisites.
app/skill-gaps/page.tsx	Skill Gap Analysis (/skill-gaps) : Categorized audit comparing current skills against career destination requirements.
app/login/ & app/signup/	Authentication Pages : Registration and login forms connected to localStorage/backend sessions.

4. Components & Backend Architecture In-Depth

frontend/components/route-map/

The Visual Graph Engine:

- `RouteMap.tsx` : Coordinates the React Flow canvas, minimap, zoom controls, and selected node state.
- `SkillNode.tsx` : Custom node component rendering status badges (completed, current, locked) and connection handles.
- `SkillDetailPanel.tsx` : Sliding flyout drawer showing skill description, estimated hours, and curated courses.
- `MapLegend.tsx` : Floating helper explaining node colors and arrow styles.

frontend/lib/ & contexts/

Data Plumbing & State Management:

- `lib/api.ts` : The API abstraction layer. All screens fetch data here. Gracefully toggles between mock data and live FastAPI calls.
- `lib/types.ts` : TypeScript interfaces for StudentProfile, Skill, Route, and GraphNode.
- `lib/mock-data/` : Realistic sample data (routes, skills, catalog) allowing full offline prototyping.
- `contexts/AuthContext.tsx` : Stores active user session state globally across all pages.

backend/services/

The Calculation Engines:

- `skill_gap.py` : Deterministic set comparison (Target Skills – Student Skills).
- `route_builder.py` : Prerequisite graph traversal calculating optimal sequence orderings.
- `rerouting.py` : Dynamic recalculation if a student changes destinations or finishes skills early.
- `opportunity_matcher.py` : Matches eligible scholarships to student skill inventories.

backend/ml/ & backend/ai/

Machine Learning & Explainable AI:

- `ml/clustering.py` : Scikit-learn K-Means clustering identifying cohort patterns across student paths.
- `ml/feature_builder.py` : Vectorizes sequential learning steps into numerical matrices.
- `ai/explainer.py` : Google Gemini integration generating clear, human explanations for why a route was recommended.

5. Core Engineering Principles Explained

1. Prerequisite Graph as a DAG (Directed Acyclic Graph)

A skill roadmap is not a simple linear list. Because some skills can be learned in parallel while others have strict dependencies, Waypoint models learning paths as a **Directed Acyclic Graph (DAG)**:

- **Directed:** Edges have arrows indicating dependency (e.g. `Python → Machine Learning`).
- **Acyclic:** There are no infinite circular dependencies (you cannot have A depend on B, B depend on C, and C depend on A).
- **Topological Sorting:** An algorithm that arranges skills in an order where all prerequisites are satisfied before subsequent skills begin.

2. Deterministic Accuracy First, AI Explanations Second

Many modern apps make the mistake of asking an LLM to "invent" a career roadmap, leading to hallucinations. Waypoint uses a hybrid design:

- **The Math is Deterministic:** The skill gaps, prerequisite order, and timelines are calculated mathematically in Python using graph theory and set operations.
- **The AI is an Explainer:** The Gemini LLM is provided the *already-calculated results* and simply explains them in encouraging, easy-to-understand language.

3. The API Abstraction Layer Pattern

In `frontend/lib/api.ts`, the frontend UI never calls raw HTTP URLs directly. Instead, UI components call functions like `getRouteGraph(id)`. This separation means the frontend can run seamlessly with mock data during UI development, and later connect to live FastAPI endpoints without changing a single line of component code.

6. Recommended Learning Roadmap (How to Build This Yourself)

If you want to master the skills needed to build platforms like Waypoint, follow this 4-phase learning progression:

Phase 1: Modern Web UI & TypeScript

1. **React Fundamentals:** Components, props, `useState`, and `useEffect`.
2. **TypeScript:** Type annotations, interfaces, and union types.
3. **Next.js 14 App Router:** Server vs. client components, page routing, layouts.
4. **Tailwind CSS:** Flexbox, CSS Grid, responsive design modifiers.

Phase 2: Graph Visualization

1. **React Flow:** Canvas setup, node positioning, edge styling.
2. **Custom Nodes:** Building custom node components with Handles.
3. **Stateful Graph Interactions:** Click handlers, zoom controls, and flyout detail panels.
4. **Framer Motion:** Smooth UI transitions and animated indicators.

Phase 3: Python Backend & APIs

1. **FastAPI:** Creating endpoints (`GET`, `POST`), path/query parameters.
2. **Pydantic v2:** Validating input/output data schemas.
3. **Pandas & CSVs:** Filtering, merging, and querying tabular datasets.
4. **CORS & HTTP:** Connecting frontend client requests to backend servers.

Phase 4: Algorithms, ML & Generative AI

1. **Graph Algorithms:** Adjacency lists, DAGs, and Topological Sorting.
2. **Scikit-learn:** Vectorization and K-Means clustering for trajectory analysis.
3. **Prompt Engineering:** Feeding structured data into the Google Gemini API to produce explainable AI summaries.